

Guia de estudio para la segunda materia

ArchUnit, JUnit4, Mockito y Vert.x
Análisis de repositorios IS-LAB-EIC-UCN y formulario de resolución

Preparado para NicoBeli

22 de junio de 2026

Resumen

Esta guía resume y operacionaliza el tipo de ejercicios que se repite en la segunda materia del curso: verificar arquitectura con ArchUnit, escribir pruebas unitarias con JUnit4 y Mockito, y migrar o integrar componentes usando Vert.x mediante EventBus. El objetivo no es memorizar código aislado, sino reconocer el patrón de examen: detectar una arquitectura degradada, formalizar reglas ejecutables, aislar dependencias externas mediante mocks y comunicar componentes con mensajería reactiva.

Índice

1. Resultado del análisis de repositorios	3
1.1. Repositorios núcleo	3
1.2. Repositorios de apoyo	3
2. Mapa mental de la segunda materia	4
3. Procedimiento general para resolver un ejercicio	4
3.1. Paso 1: dibujar paquetes y dependencias	4
3.2. Paso 2: proponer arquitectura corregida	5
3.3. Paso 3: elegir el patrón Vert.x correcto	5
4. Dependencias Maven mínimas	5
5. ArchUnit: formulario de reglas	6
5.1. Estructura de un test ArchUnit con JUnit4	6
5.2. Patrón S-R-C-E-X para escribir reglas	6
5.3. Reglas recurrentes	7
5.3.1. Controller no accede a repositorio ni infraestructura	7
5.3.2. Dominio puro	7
5.3.3. Service/Application no depende de UI	7
5.3.4. Repositorios concretos no son conocidos por capas superiores	7
5.3.5. Arquitectura por capas	7
5.3.6. Arquitectura hexagonal/onion	8
5.3.7. Slices sin ciclos	8
5.4. Checklist ArchUnit antes de entregar	8
6. JUnit4: formulario de pruebas unitarias	8
6.1. Estructura canónica	8
6.2. Qué testear con JUnit4 puro	9
6.3. Pruebas parametrizadas	9

6.4. Reglas de calidad para tests	10
7. Mockito: formulario de mocks	10
7.1. Cuando usar Mockito	10
7.2. Patron Arrange-Act-Assert con Mockito	11
7.3. Verificar que algo no ocurre	11
7.4. Mock de EntityManager y TypedQuery	12
7.5. ArgumentCaptor	12
8. Vert.x: formulario de EventBus	13
8.1. Regla de oro	13
8.2. Point-to-Point con send()	13
8.3. Publish/Subscribe con publish()	13
8.4. Request/Response con request() y reply()	14
8.5. Plantilla de main	15
9. Como decidir entre los tres patrones Vert.x	15
10.Recetas por tipo de ejercicio	16
10.1. Ejercicio tipo MVC degradado	16
10.2. Ejercicio tipo monitoreo ambiental	17
10.3. Ejercicio tipo Pipe-and-Filter	17
11.Erroros frecuentes y correccion	18
12.Plan de estudio sugerido	18
12.1. Bloque 1: arquitectura y ArchUnit	18
12.2. Bloque 2: JUnit4 y Mockito	18
12.3. Bloque 3: Vert.x	19
12.4. Bloque 4: simulacion de examen	19
13.Checklist final de entrega	19
13.1. ArchUnit	19
13.2. JUnit4	19
13.3. Mockito	20
13.4. Vert.x	20
14.Mini-formulario de codigo	20
14.1. ArchUnit imports frecuentes	20
14.2. JUnit4 imports frecuentes	20
14.3. Mockito imports frecuentes	20
14.4. Vert.x imports frecuentes	21
15.Fuentes consultadas	21
15.1. Repositorios GitHub	21
15.2. Material de clases usado como base conceptual	21

1 Resultado del analisis de repositorios

El repositorio organizacional contiene varios proyectos, pero no todos corresponden al nucleo de la segunda materia. Para esta guia clasifique los repositorios segun tres criterios:

1. Que pidan o sugieran **ArchUnit** para verificar reglas de arquitectura.
2. Que pidan **JUnit4** y, de preferencia, **Mockito** para testear unidades con dependencias simuladas.
3. Que pidan **Vert.x** con algun patron de EventBus: Point-to-Point, Publish/Subscribe o Request/Response.

1.1 Repositorios nucleo

Repositorio	Por que importa	Que debes extraer para estudiar
FormativoII-PSP-2026	Guia formativa mas cercana a la prueba actual. Integra arquitectura degradada, ArchUnit, JUnit4, Mockito, Vert.x y cobertura JaCoCo.	Es el modelo principal: controller no debe depender de infraestructura; la coordinacion debe moverse a application/service; Mockito se usa para servicio externo, repositorio, notificador y publicador de eventos; Vert.x se usa para publicar eventos de dominio.
Evaluacion-II-II-2025	Evaluacion formal de segunda materia con sistema de monitoreo ambiental.	Reglas por capas con ArchUnit; pruebas con EntityManager y TypedQuery mockeados; Vert.x Publish/Subscribe para mediciones ambientales.
Formativa-II-II-2025	Formativa orientada a JPA, Mockito, ArchUnit, JUnit y migracion parcial a Vert.x.	Buen repositorio para practicar dependencias permitidas/prohibidas, tests de servicio sin base de datos real y transformacion hacia EventBus.
FormativaII-2025	Version anterior con capas, JUnit4+Mockito y Vert.x Publish/Subscribe.	Refuerza el mismo patron: dominio puro, servicio desacoplado de implementaciones concretas y eventos por canales.
EvaluacionII-I-2025	Evaluacion con arquitectura Pipe-and-Filter, ArchUnit, JUnit4+Mockito y Vert.x Pub/Sub.	Muy util para entender cuando cada filtro se transforma en un Verticle y el flujo se vuelve una cadena de canales del EventBus.

1.2 Repositorios de apoyo

Repositorio	Uso recomendado
bancoUCN	Practicar arquitectura hexagonal y reglas ArchUnit. No es un ejercicio completo de Vert.x, pero es excelente para aprender dominio, puertos, adaptadores y reglas contra dependencias indebidas.
VertX	Referencia directa de los tres patrones: Point-to-Point, Publish/Subscribe y Request/Response. Usalo como banco de snippets.

Repositorio	Uso recomendado
<code>ejercicioTesting</code>	Base para JUnit4: particiones, casos validos/invalidos, asserts, fixture y estructura de pruebas.
<code>Laboratorio-II-II-2025</code>	Complementario para diseno de casos, pruebas parametrizadas y casos borde. No es el nucleo si la pregunta exige ArchUnit y Vert.x.

2 Mapa mental de la segunda materia

La segunda materia combina tres dimensiones que suelen aparecer juntas:

Dimension	Pregunta que debes responder
Arquitectura	Quien puede depender de quien? Que dependencia viola la arquitectura planeada?
Testing unitario	Como pruebo la regla de negocio sin base de datos, red, UI, Vert.x ni servicios externos reales?
Reactividad	Que interaccion debe ser un evento, una tarea de un solo consumidor o una solicitud con respuesta?

La respuesta de examen normalmente debe tener esta forma:

1. Detectar la violacion arquitectural.
2. Refactorizar hacia una capa de aplicacion o servicio.
3. Introducir interfaces cuando una dependencia externa rompe el desacoplamiento.
4. Escribir reglas ArchUnit que hagan fallar el proyecto si la violacion reaparece.
5. Escribir tests JUnit4 de dominio y servicio.
6. Usar Mockito para simular repositorios, servicios externos, notificaciones o publicadores.
7. Integrar Vert.x dejando la regla de negocio fuera del Verticle.

3 Procedimiento general para resolver un ejercicio

3.1 Paso 1: dibujar paquetes y dependencias

Antes de escribir codigo, identifica los paquetes. Una estructura tipica es:

```

cl.ucn.proyecto
  controller | web | view
  application | service | caso_uso
  domain | model
  repository | dao
  infrastructure | infra | persistence | external
  vertx | messaging
  
```

Luego marca dependencias prohibidas. En los ejercicios observados, las violaciones tipicas son:

- `controller` depende directamente de `repository`, `dao`, `infrastructure` o `persistence`.
- `domain` depende de JPA, Hibernate, Spring, Vert.x, JavaFX, Javalin o clases de infraestructura.
- `service` depende de `controller`, `view` o detalles de UI.

- Un Verticle contiene reglas de negocio o accede directamente a la base de datos.
- Un servicio depende de una clase concreta de repositorio en vez de una interfaz.

3.2 Paso 2: proponer arquitectura corregida

La forma corregida mas frecuente es:

```
View/Controller -> Application/Service -> Domain
                    |
                    v
                Port/Repository interface
                    ^
                    |
                Infrastructure/Persistence adapter

Messaging/Verticle -> Application/Service
```

Regla conceptual: las capas externas pueden conocer capas internas; el nucleo no debe conocer detalles externos.

3.3 Paso 3: elegir el patron Vert.x correcto

Necesidad	Metodo EventBus	Interpretacion
Un solo receptor debe procesar la tarea	<code>send()</code>	Point-to-Point. Sirve para trabajo distribuido o balance de carga.
Todos los interesados deben enterarse	<code>publish()</code>	Publish/Subscribe. Sirve para eventos de dominio y notificaciones.
El emisor necesita una respuesta	<code>request() + reply()</code>	Request/Response. Sirve para consultas o validaciones tipo RPC no bloqueante.

4 Dependencias Maven minimas

Los repositorios de esta materia usan Maven. La version exacta puede variar, pero el patron de dependencias es estable. Para JUnit4 y ArchUnit con runner JUnit4, usa:

```
<dependencies>
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>4.11.0</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>com.tngtech.archunit</groupId>
```

```

    <artifactId>archunit-junit4</artifactId>
    <version>1.3.0</version>
    <scope>test</scope>
</dependency>

<dependency>
    <groupId>io.vertx</groupId>
    <artifactId>vertx-core</artifactId>
    <version>4.5.9</version>
</dependency>
</dependencies>

```

Para cobertura, JaCoCo suele configurarse en el ciclo de Maven. Si el enunciado exige 80 %, no basta con crear tests: debes ejecutar `mvn clean test` o `mvn clean verify` y revisar el reporte.

5 ArchUnit: formulario de reglas

5.1 Estructura de un test ArchUnit con JUnit4

```

import com.tngtech.archunit.core.importer.ImportOption;
import com.tngtech.archunit.junit.AnalyzeClasses;
import com.tngtech.archunit.junit.ArchTest;
import com.tngtech.archunit.junit.ArchUnitRunner;
import com.tngtech.archunit.lang.ArchRule;
import org.junit.runner.RunWith;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

@RunWith(ArchUnitRunner.class)
@AnalyzeClasses(
    packages = "cl.ucn.proyecto",
    importOptions = ImportOption.DoNotIncludeTests.class
)
public class ArchitectureTest {

    @ArchTest
    public static final ArchRule controller_no_depende_de_infra =
        noClasses().that().resideInAPackage("..controller..")
            .should().dependOnClassesThat()
                .resideInAnyPackage("..repository..", "..dao..", "..infrastructure..", "..
persistence..")
            .because("el controlador debe delegar casos de uso a application/service");
}

```

5.2 Patron S-R-C-E-X para escribir reglas

Scope Paquete base importado por `@AnalyzeClasses`.

Refiere Sujeto seleccionado con `classes().that()` o `noClasses().that()`.

Condicion Lo permitido o prohibido con `should()`.

Evidencia Razon pedagogica en `because()`.

Xcepciones Solo si el enunciado permite deuda tecnica o casos legacy.

5.3 Reglas recurrentes

5.3.1 Controller no accede a repositorio ni infraestructura

```
@ArchTest
public static final ArchRule controller_no_accede_a_persistencia =
    noClasses().that().resideInAnyPackage("..controller..", "..web..", "..view..")
        .should().dependOnClassesThat()
            .resideInAnyPackage("..repository..", "..dao..", "..persistence..", "..infra..")
            .because("la entrada del sistema debe hablar con casos de uso, no con la base de
datos");
```

5.3.2 Dominio puro

```
@ArchTest
public static final ArchRule dominio_no_conoce_frameworks =
    noClasses().that().resideInAPackage("..domain..")
        .should().dependOnClassesThat()
            .resideInAnyPackage(
                "..controller..", "..view..", "..web..",
                "..repository..", "..dao..", "..persistence..", "..infra..",
                "io.vertx..", "org.hibernate..", "jakarta.persistence..", "javax.persistence..",
                "org.springframework..", "io.javalin..")
            .because("el dominio debe expresar reglas de negocio sin detalles tecnicos");
```

5.3.3 Service/Application no depende de UI

```
@ArchTest
public static final ArchRule application_no_depends_de_interfaz =
    noClasses().that().resideInAnyPackage("..application..", "..service..")
        .should().dependOnClassesThat()
            .resideInAnyPackage("..controller..", "..view..", "..web..", "javafx..")
            .because("la aplicacion coordina casos de uso y no debe conocer la interfaz");
```

5.3.4 Repositorios concretos no son conocidos por capas superiores

```
@ArchTest
public static final ArchRule servicios_no_dependen_de_implementaciones_concretas =
    noClasses().that().resideInAnyPackage("..service..", "..application..")
        .should().dependOnClassesThat()
            .resideInAnyPackage("..repository.impl..", "..dao.impl..", "..persistence..")
            .because("los casos de uso deben depender de interfaces, no de adaptadores
concretos");
```

5.3.5 Arquitectura por capas

```
import static com.tngtech.archunit.library.Architectures.layeredArchitecture;

@ArchTest
public static final ArchRule capas = layeredArchitecture()
    .consideringAllDependencies()
```

```
.layer("Controller").definedBy("..controller..", "..web..")
.layer("Service").definedBy("..service..", "..application..")
.layer("Repository").definedBy("..repository..", "..dao..", "..persistence..")
.whereLayer("Controller").mayNotBeAccessedByAnyLayer()
.whereLayer("Service").mayOnlyBeAccessedByLayers("Controller")
.whereLayer("Repository").mayOnlyBeAccessedByLayers("Service");
```

5.3.6 Arquitectura hexagonal/onion

```
import static com.tngtech.archunit.library.Architectures.onionArchitecture;

@ArchTest
public static final ArchRule arquitectura_hexagonal = onionArchitecture()
    .domainModels("..domain.model..", "..domain..")
    .domainServices("..domain.service..")
    .applicationServices("..application..", "..service..")
    .adapter("web", "..adapter.in.web..", "..controller..")
    .adapter("persistence", "..adapter.out.persistence..", "..persistence..")
    .adapter("messaging", "..adapter.out.messaging..", "..vertx..", "..messaging..")
    .because("las dependencias deben apuntar hacia el nucleo del negocio");
```

5.3.7 Slices sin ciclos

```
import static com.tngtech.archunit.library.dependencies.SlicesRuleDefinition.slices;

@ArchTest
public static final ArchRule features_sin_ciclos =
    slices().matching("cl.ucn.proyecto.(*)..")
        .should().beFreeOfCycles()
        .because("los ciclos impiden evolucion y pruebas independientes");
```

5.4 Checklist ArchUnit antes de entregar

- ☐ El paquete en `@AnalyzeClasses` coincide con el paquete real.
- ☐ La regla usa `noClasses()` si quieres prohibir dependencias.
- ☐ La regla usa `classes()` si quieres obligar una propiedad.
- ☐ Incluiste `because()` con justificacion arquitectural.
- ☐ No escribiste una regla tan amplia que impida dependencias legitimas como `java...`
- ☐ Ejecutaste `mvn test` y viste que la regla falla antes del refactor y pasa despues.

6 JUnit4: formulario de pruebas unitarias

6.1 Estructura canonica

```
import org.junit.Before;
import org.junit.Test;

import static org.junit.Assert.*;
```



```
public class SolicitudTest {

    private Solicitud solicitud;

    @Before
    public void setUp() {
        solicitud = new Solicitud("alumno@ucn.cl", "Certificado de alumno regular");
    }

    @Test
    public void solicitudNuevaPartePendiente() {
        assertEquals(EstadoSolicitud.PENDIENTE, solicitud.getEstado());
    }

    @Test
    public void aprobarSolicitudCambiaEstado() {
        solicitud.aprobar();
        assertEquals(EstadoSolicitud.APROBADA, solicitud.getEstado());
    }

    @Test(expected = IllegalStateException.class)
    public void noSePuedeRechazarUnaSolicitudAprobada() {
        solicitud.aprobar();
        solicitud.rechazar("documentacion incompleta");
    }
}
```

6.2 Que testear con JUnit4 puro

JUnit4 puro se usa cuando no hay dependencias externas. Ejemplos:

- Validaciones de constructor.
- Transiciones de estado.
- Calculos de promedio, totales, descuentos o clasificaciones.
- Particiones validas e invalidas.
- Casos borde: lista vacia, nulos, duplicados, limites numericos, fechas extremas.

6.3 Pruebas parametrizadas

Utiles cuando el enunciado da varias entradas equivalentes.

```
import org.junit.Test;
import org.junit.runner.RunWith;
import org.junit.runners.Parameterized;

import java.util.Arrays;
import java.util.Collection;

import static org.junit.Assert.assertEquals;

@RunWith(Parameterized.class)
public class IdentifierValidatorTest {

    @Parameterized.Parameters(name = "{index}: {0} -> {1}")
```

```
public static Collection<Object[]> data() {
    return Arrays.asList(new Object[][] {
        {"abc", true},
        {"a1", true},
        {"1abc", false},
        {"", false},
        {"a-b", false}
    });
}

private final String input;
private final boolean esperado;

public IdentifierValidatorTest(String input, boolean esperado) {
    this.input = input;
    this.esperado = esperado;
}

@Test
public void validaIdentificador() {
    assertEquals(esperado, IdentifierValidator.isValid(input));
}
}
```

6.4 Reglas de calidad para tests

- **Fast:** no base de datos real, no red, no esperas arbitrarias.
- **Independent:** cada test crea su propio fixture.
- **Repeatable:** mismo resultado en cualquier ejecucion.
- **Self-checking:** tiene asserts o verificaciones claras.
- **Timely:** se escriben cerca del codigo que validan.

7 Mockito: formulario de mocks

7.1 Cuando usar Mockito

Usa Mockito cuando la clase bajo test depende de algo que no quieres ejecutar realmente:

- Repositorio o DAO.
- EntityManager o TypedQuery.
- Servicio remoto.
- Notificador por correo.
- Publicador de eventos.
- Gateway externo.

La idea no es “hacer trampa”, sino mantener el test como unidad: pruebas la logica del servicio, no la base de datos ni la red.

7.2 Patron Arrange-Act-Assert con Mockito

```
import org.junit.Before;
import org.junit.Test;

import static org.junit.Assert.*;
import static org.mockito.ArgumentMatchers.*;
import static org.mockito.Mockito.*;

public class SolicitudServiceTest {

    private SolicitudRepository repository;
    private Notificador notificador;
    private EventPublisher eventPublisher;
    private SolicitudService service;

    @Before
    public void setUp() {
        repository = mock(SolicitudRepository.class);
        notificador = mock(Notificador.class);
        eventPublisher = mock(EventPublisher.class);
        service = new SolicitudService(repository, notificador, eventPublisher);
    }

    @Test
    public void aprobarSolicitudGuardaNotificaYPublicaEvento() {
        Solicitud solicitud = new Solicitud(10L, "nico@ucn.cl", "Certificado");
        when(repository.findById(10L)).thenReturn(java.util.Optional.of(solicitud));

        service.aprobar(10L);

        assertEquals(EstadoSolicitud.APROBADA, solicitud.getEstado());
        verify(repository).save(solicitud);
        verify(notificador).enviar(eq("nico@ucn.cl"), contains("aprobada"));
        verify(eventPublisher).publish(any(SolicitudAprobadaEvent.class));
    }
}
```

7.3 Verificar que algo no ocurre

Esto aparece en escenarios de falla: si una operacion no se completa, no debe notificarse ni publicarse evento.

```
@Test(expected = IllegalArgumentException.class)
public void siSolicitudNoExisteNoNotificaNiPublica() {
    when(repository.findById(99L)).thenReturn(java.util.Optional.empty());

    try {
        service.aprobar(99L);
    } finally {
        verify(repository, never()).save(any(Solicitud.class));
        verifyNoInteractions(notificador);
        verifyNoInteractions(eventPublisher);
    }
}
```

7.4 Mock de EntityManager y TypedQuery

En ejercicios con JPA, no debes levantar SQLite ni una base real para un test unitario de servicio. Debes simular la consulta.

```
import jakarta.persistence.EntityManager;
import jakarta.persistence.TypedQuery;
import org.junit.Test;

import java.util.Arrays;
import java.util.List;

import static org.junit.Assert.assertEquals;
import static org.mockito.ArgumentMatchers.*;
import static org.mockito.Mockito.*;

public class SensorServiceImplTest {

    @Test
    public void calculaPromedioDeMedicionesDelSensor() {
        EntityManager em = mock(EntityManager.class);
        TypedQuery<Measurement> query = mock(TypedQuery.class);

        when(em.createQuery(anyString(), eq(Measurement.class))).thenReturn(query);
        when(query.setParameter(eq("sensorId"), eq(7L))).thenReturn(query);

        List<Measurement> mediciones = Arrays.asList(
            new Measurement(7L, 20.0),
            new Measurement(7L, 22.0),
            new Measurement(7L, 24.0)
        );
        when(query.getResultList()).thenReturn(mediciones);

        SensorService service = new SensorServiceImpl(em);

        double promedio = service.averageTemperature(7L);

        assertEquals(22.0, promedio, 0.001);
        verify(query).getResultList();
    }
}
```

7.5 ArgumentCaptor

Si necesitas revisar exactamente que objeto fue guardado o publicado:

```
import org.mockito.ArgumentCaptor;

@Test
public void publicarEventoIncluyeIdYEstado() {
    Solicitud solicitud = new Solicitud(5L, "a@ucn.cl", "Certificado");
    when(repository.findById(5L)).thenReturn(java.util.Optional.of(solicitud));

    service.aprobar(5L);

    ArgumentCaptor<SolicitudAprobadaEvent> captor =
        ArgumentCaptor.forClass(SolicitudAprobadaEvent.class);
    verify(eventPublisher).publish(captor.capture());
}
```

```
assertEquals(Long.valueOf(5L), captor.getValue().getSolicitudId());
assertEquals(EstadoSolicitud.APROBADA, captor.getValue().getEstado());
}
```

8 Vert.x: formulario de EventBus

8.1 Regla de oro

Un Verticle no deberia concentrar reglas de negocio. El Verticle traduce mensajes, valida formato minimo y delega al servicio de aplicacion. La logica importante se testea con JUnit4 y Mockito.

```
Incorrecto:
Verticle -> calcula regla de negocio -> EntityManager -> save

Correcto:
Verticle -> parsea JsonObject -> ApplicationService -> Repository/Ports
```

8.2 Point-to-Point con send()

Uso: una tarea debe ser procesada por un solo consumidor. Si hay varios consumidores en la misma direccion, Vert.x distribuye.

```
import io.vertx.core.AbstractVerticle;
import io.vertx.core.Promise;

public class TaskSenderVerticle extends AbstractVerticle {
    public static final String ADDRESS = "tasks.process";

    @Override
    public void start(Promise<Void> startPromise) {
        startPromise.complete();

        vertx.setPeriodic(1000, id -> {
            vertx.eventBus().send(ADDRESS, "procesar tarea");
        });
    }
}

public class TaskWorkerVerticle extends AbstractVerticle {
    @Override
    public void start(Promise<Void> startPromise) {
        vertx.eventBus().consumer(TaskSenderVerticle.ADDRESS, message -> {
            System.out.println("Worker recibio: " + message.body());
        });
        startPromise.complete();
    }
}
```

8.3 Publish/Subscribe con publish()

Uso: evento de dominio o notificacion global. Todos los consumidores suscritos reciben copia.

```

import io.vertx.core.AbstractVerticle;
import io.vertx.core.Promise;
import io.vertx.core.json.JsonObject;

public class SolicitudEventPublisherVerticle extends AbstractVerticle {
    public static final String ADDRESS = "solicitudes.eventos";

    @Override
    public void start(Promise<Void> startPromise) {
        startPromise.complete();

        JsonObject evento = new JsonObject()
            .put("solicitudId", 10)
            .put("estado", "APROBADA")
            .put("email", "nico@ucn.cl");

        vertx.eventBus().publish(ADDRESS, evento);
    }
}

public class AuditoriaSubscriberVerticle extends AbstractVerticle {
    @Override
    public void start(Promise<Void> startPromise) {
        vertx.eventBus().<JsonObject>consumer(SolicitudEventPublisherVerticle.ADDRESS, message
        -> {
            JsonObject evento = message.body();
            System.out.println("Auditoria: " + evento.encode());
        });
        startPromise.complete();
    }
}

public class NotificacionSubscriberVerticle extends AbstractVerticle {
    @Override
    public void start(Promise<Void> startPromise) {
        vertx.eventBus().<JsonObject>consumer(SolicitudEventPublisherVerticle.ADDRESS, message
        -> {
            JsonObject evento = message.body();
            System.out.println("Notificar a: " + evento.getString("email"));
        });
        startPromise.complete();
    }
}

```

8.4 Request/Response con request() y reply()

Uso: el emisor necesita respuesta. No bloquea el hilo: la respuesta llega por callback.

```

import io.vertx.core.AbstractVerticle;
import io.vertx.core.Promise;
import io.vertx.core.json.JsonObject;

public class SolicitudQueryVerticle extends AbstractVerticle {
    public static final String ADDRESS = "solicitudes.consultar";

    @Override
    public void start(Promise<Void> startPromise) {
        startPromise.complete();
    }
}

```

```

        JsonObject request = new JsonObject().put("solicitudId", 10);
        vertx.eventBus().<JsonObject>request(ADDRESS, request, reply -> {
            if (reply.succeeded()) {
                System.out.println("Respuesta: " + reply.result().body().encode());
            } else {
                System.err.println("Fallo: " + reply.cause().getMessage());
            }
        });
    }
}

public class SolicitudQueryHandlerVerticle extends AbstractVerticle {
    private final SolicitudService service;

    public SolicitudQueryHandlerVerticle(SolicitudService service) {
        this.service = service;
    }

    @Override
    public void start(Promise<Void> startPromise) {
        vertx.eventBus().<JsonObject>consumer(SolicitudQueryVerticle.ADDRESS, message -> {
            Long id = message.body().getLong("solicitudId");
            SolicitudDTO dto = service.buscar(id);

            JsonObject response = new JsonObject()
                .put("id", dto.getId())
                .put("estado", dto.getEstado());
            message.reply(response);
        });
        startPromise.complete();
    }
}

```

8.5 Plantilla de main

```

import io.vertx.core.Vertx;
import io.vertx.core.DeploymentOptions;

public class Main {
    public static void main(String[] args) {
        Vertx vertx = Vertx.vertx();

        vertx.deployVerticle(new TaskSenderVerticle());
        vertx.deployVerticle(TaskWorkerVerticle.class.getName(),
            new DeploymentOptions().setInstances(2));

        vertx.deployVerticle(new AuditoriaSubscriberVerticle());
        vertx.deployVerticle(new NotificacionSubscriberVerticle());
    }
}

```

9 Como decidir entre los tres patrones Vert.x

Situacion del enunciado	Patron	Metodo clave	Error comun
“Procesar cada medicion una sola vez”	Point-to-Point	<code>send()</code>	Usar <code>publish()</code> y duplicar procesamiento.
“Notificar a auditoria, UI y correo”	Publish/Subscribe	<code>publish()</code>	Usar <code>send()</code> y que solo un consumidor reciba.
“Consultar promedio y devolver resultado”	Request/Response	<code>request()</code> y <code>reply()</code>	Bloquear con esperas o no manejar fallo.
“Transformar filtros en pipeline”	Pub/Sub o cadena send/publish segun enunciado	Canales por etapa	Llamar filtros directamente en vez de EventBus.

10 Recetas por tipo de ejercicio

10.1 Ejercicio tipo MVC degradado

Sintoma: **Controller** accede directamente a repositorios, servicios externos o clases de infraestructura.

Solucion esperada:

1. Crear un caso de uso en **application** o **service**.
2. Mover ahi la coordinacion: buscar entidad, validar, cambiar estado, guardar, notificar, publicar evento.
3. El controller queda delgado: parsea input, llama al servicio, prepara respuesta.
4. Crear interfaces para dependencias externas.
5. Agregar reglas ArchUnit para impedir que controller vuelva a depender de infraestructura.

Plantilla:

```
public class SolicitudController {
    private final SolicitudApplicationService service;

    public SolicitudController(SolicitudApplicationService service) {
        this.service = service;
    }

    public void aprobar(Long id) {
        service.aprobarSolicitud(id);
    }
}

public class SolicitudApplicationService {
    private final SolicitudRepository repository;
    private final Notificador notificador;
    private final EventPublisher publisher;

    public SolicitudApplicationService(SolicitudRepository repository,
                                       Notificador notificador,
                                       EventPublisher publisher) {
        this.repository = repository;
        this.notificador = notificador;
    }
}
```



```

    this.publisher = publisher;
}

public void aprobarSolicitud(Long id) {
    Solicitud solicitud = repository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("No existe solicitud"));
    solicitud.aprobar();
    repository.save(solicitud);
    notificador.enviar(solicitud.getEmail(), "Solicitud aprobada");
    publisher.publish(new SolicitudAprobadaEvent(id, solicitud.getEmail()));
}
}

```

10.2 Ejercicio tipo monitoreo ambiental

Sintoma: proyecto con sensores, mediciones, repositorio y verticles. Se pide promedio, registro de mediciones y eventos.

Solucion esperada:

- Servicio calcula promedio usando repositorio o EntityManager mockeable.
- Verticle consumidor recibe `JsonObject`, valida campos y llama al servicio.
- Se usa Publish/Subscribe para que multiples consumidores reaccionen a una medicion.
- ArchUnit protege que `web` no dependa de `repository` y que `repository` no dependa de `web`.

Canales sugeridos:

```

air.monitor.measurements
sensor.measurement.raw
sensor.measurement.validated
sensor.measurement.alerts

```

10.3 Ejercicio tipo Pipe-and-Filter

Sintoma: filtros encadenados que antes se llamaban directamente y ahora deben funcionar con Vert.x.

Solucion esperada:

```
raw-data -> filtro-validacion -> filtro-normalizacion -> filtro-alerta -> salida
```

Cada filtro puede ser un Verticle:

```

public class ValidationFilterVerticle extends AbstractVerticle {
    public static final String INPUT = "raw.data";
    public static final String OUTPUT = "validated.data";

    private final RawDataFilter filter;

    public ValidationFilterVerticle(RawDataFilter filter) {
        this.filter = filter;
    }

    @Override
    public void start(Promise<Void> startPromise) {
        vertx.eventBus().<JsonObject>consumer(INPUT, message -> {
            JsonObject validated = filter.apply(message.body());

```

```

    vertx.eventBus().publish(OUTPUT, validated);
  });
  startPromise.complete();
}
}

```

11 Errores frecuentes y correccion

Error	Por que esta mal	Correccion
Controller inyecta JpaRepository.	Salta application/service y acopla entrada con persistencia.	Controller inyecta caso de uso; caso de uso depende de interfaz.
Dominio tiene @Entity.	Mezcla regla de negocio con JPA si el enunciado exige dominio puro.	Crear entidad de persistencia separada o mapper.
Test unitario abre SQLite.	Ya no es unitario; es integracion y mas lento.	Mock de repositorio, EntityManager o TypedQuery.
Verticle calcula reglas de negocio.	Dificulta testeo y acopla mensajeria con dominio.	Verticle delega a servicio.
Usar send() para notificar varios subscriptores.	Solo uno recibira el mensaje.	Usar publish().
Usar publish() para una tarea que debe ejecutarse una vez.	Duplica trabajo si hay varios consumidores.	Usar send().
No verificar interacciones Mockito.	El test puede pasar aunque no se guarde ni notifique.	Usar verify(), never(), ArgumentCaptor.
ArchUnit con paquete equivocado.	La regla no analiza tus clases reales.	Revisar package base y ejecutar test con una violacion conocida.

12 Plan de estudio sugerido

12.1 Bloque 1: arquitectura y ArchUnit

1. Leer bancoUCN y dibujar dominio, aplicacion, puertos y adaptadores.
2. Escribir tres reglas: dominio puro, controller no accede a persistencia, servicio no depende de UI.
3. Romper intencionalmente una regla y comprobar que el test falla.
4. Arreglar la dependencia y comprobar que pasa.

12.2 Bloque 2: JUnit4 y Mockito

1. Hacer tests JUnit4 puros para entidades de dominio.
2. Hacer tests de servicio con repositorio mockeado.
3. Agregar test de fallo con never() y verifyNoInteractions().

4. Practicar mock de `EntityManager` y `TypedQuery`.

12.3 Bloque 3: Vert.x

1. Ejecutar el repositorio `VertX`.
2. Reescribir los tres patrones sin mirar el código.
3. Integrar un servicio propio en un `Verticle`.
4. Verificar que la regla de negocio sigue testada con `JUnit4`, no solo por ejecución manual del `Verticle`.

12.4 Bloque 4: simulacion de examen

Toma `FormativoII-PSP-2026` o `Evaluacion-II-II-2025` y resuelve en este orden:

1. `mvn test`: ver fallas iniciales.
2. Crear o corregir `ArchitectureTest`.
3. Refactorizar `controller/service/repository`.
4. Agregar tests `JUnit4` de dominio.
5. Agregar tests `Mockito` de servicio.
6. Implementar `Vert.x` según el patrón pedido.
7. Ejecutar `mvn clean verify` y revisar cobertura.

13 Checklist final de entrega

13.1 ArchUnit

- ☐ Existe clase `ArchitectureTest`.
- ☐ Usa `@RunWith(ArchUnitRunner.class)` si el proyecto es `JUnit4`.
- ☐ Usa `@AnalyzeClasses(packages = "...")`.
- ☐ Tiene reglas para dependencia `controller/service/repository` o hexagonal.
- ☐ Tiene reglas de dominio puro si el enunciado lo exige.
- ☐ Los tests fallan si introduces la violación descrita.

13.2 JUnit4

- ☐ Tests con `@Test` y asserts claros.
- ☐ Casos válidos, inválidos y borde.
- ☐ Sin base de datos real en tests unitarios.
- ☐ Fixture en `@Before` si se repite configuración.
- ☐ Excepciones testadas con `expected` o `try/catch`.

13.3 Mockito

- ☐ Dependencias externas reemplazadas por mocks.
- ☐ `when(...).thenReturn(...)` para preparar comportamiento.
- ☐ `verify(...)` para guardar, publicar o notificar.
- ☐ `never()` o `verifyNoInteractions()` para escenarios de error.
- ☐ `ArgumentCaptor` si importa el contenido exacto del objeto enviado.

13.4 Vert.x

- ☐ Elegiste `send()`, `publish()` o `request()` segun el requerimiento.
- ☐ Direcciones del `EventBus` son constantes y coherentes.
- ☐ Mensajes estructurados con `JsonObject` si hay varios campos.
- ☐ Vertices no contienen reglas de negocio complejas.
- ☐ Se despliegan los Vertices necesarios en `main`.

14 Mini-formulario de codigo

14.1 ArchUnit imports frecuentes

```
import com.tngtech.archunit.junit.AnalyzeClasses;
import com.tngtech.archunit.junit.ArchTest;
import com.tngtech.archunit.junit.ArchUnitRunner;
import com.tngtech.archunit.lang.ArchRule;
import org.junit.runner.RunWith;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;
import static com.tngtech.archunit.library.Architectures.layeredArchitecture;
import static com.tngtech.archunit.library.Architectures.onionArchitecture;
import static com.tngtech.archunit.library.dependencies.SlicesRuleDefinition.slices;
```

14.2 JUnit4 imports frecuentes

```
import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;
```

14.3 Mockito imports frecuentes

```
import static org.mockito.Mockito.*;
import static org.mockito.ArgumentMatchers.*;
import org.mockito.ArgumentCaptor;
```

14.4 Vert.x imports frecuentes

```
import io.vertx.core.AbstractVerticle;  
import io.vertx.core.Promise;  
import io.vertx.core.Vertx;  
import io.vertx.core.DeploymentOptions;  
import io.vertx.core.eventbus.EventBus;  
import io.vertx.core.json.JsonObject;
```

15 Fuentes consultadas

15.1 Repositorios GitHub

- <https://github.com/IS-LAB-EIC-UCN/FormativoII-PSP-2026>
- <https://github.com/IS-LAB-EIC-UCN/Evaluacion-II-II-2025>
- <https://github.com/IS-LAB-EIC-UCN/Formativa-II-II-2025>
- <https://github.com/IS-LAB-EIC-UCN/FormativaII-2025>
- <https://github.com/IS-LAB-EIC-UCN/EvaluacionII-I-2025>
- <https://github.com/IS-LAB-EIC-UCN/bancoUCN>
- <https://github.com/IS-LAB-EIC-UCN/VertX>
- <https://github.com/IS-LAB-EIC-UCN/ejercicioTesting>
- <https://github.com/IS-LAB-EIC-UCN/Laboratorio-II-II-2025>

15.2 Material de clases usado como base conceptual

- *ArchUnit - Chequeo arquitectural automatizado en Java y validacion de arquitectura hexagonal.*
- *Testing de Sistemas.*
- *Introduccion a la Programacion Reactiva.*
- *Arquitectura de Software.*